Latent Cognitive Blocks: Parallel Causal Span Control for Just-in-Time Coprocessor Invocation

Working Draft

August 2026

Abstract

Explicit cognitive blocks serialize control through language tokens. We pre-register a parallel causal channel that marks the start, coarse type, and end of a cognitive-assistance span while ordinary autoregressive generation continues. The stable controller vocabulary is NONE, COMPUTE, RETRIEVE, VERIFY, and HELP; a second-stage runtime resolves deployment-specific capabilities. This is a protocol paper, not a positive latent-control result. Paper 2's engine-specific learned-routing gate remains `no_go`. The independent latent experiment may proceed only as a matched comparison against generic paired tags, fenced blocks, label-only intent, transparent CPU triggers, and oracle spans. We separate causal span control from payload normalization, deterministic execution, result integration, and optional PRA materialization.

1 Status and Motivation

Paper 2 shows that deterministic parsing and execution are reliable once a capability is known, but learned engine identity is unsafe under held-out controls and registry growth. Its machine-readable Paper 3 gate is `no_go`; that decision is not overwritten here. The new question is narrower and registry-independent: can a frozen Transformer expose when a span needs cognitive assistance, what coarse assistance family it needs, and when the span is complete, without serializing control syntax into generated text?

The hypothesis concerns transport across the neural–runtime boundary, not a larger engine classifier. Conventional explicit tools remain the universal fallback. Runtime-copy remains the default for authoritative exact outputs.

2 Factorized Architecture

At token t and selected layer l, a frozen representation $h_t^{(l)}$ feeds a small causal controller. Outside a block,

$$p_{\text{start}}^{(l)} = \text{softmax}(W_{\text{start}}^{(l)} h_t^{(l)})$$

ranges over NONE, COMPUTE, RETRIEVE, VERIFY, and HELP. Inside a block, a separate head predicts CONTINUE or END. No future token may inform a headline decision.

The two routing stages are

$$R_1 : \text{generation} \to \{\text{NONE}, \text{COMPUTE}, \text{RETRIEVE}, \text{VERIFY}, \text{HELP}\},$$

$$R_2 : (\text{intent}, \text{span}, \text{registry}, \text{policy}, \text{state}) \to \text{capability or shortlist}.$$

R_1 never predicts calculator, graph, Datalog, DuckDB, or another registry identity. R_2 may use deterministic rules, token-aware BM25, a CPU classifier, capability retrieval, optional PRA, or an explicit tool fallback.

2.1 Constrained state machine

The preferred minimal machine has two states:

- **OUTSIDE**: emit NONE or $\text{START}_{\{\text{compute,retrieve,verify,help}\}}$;
- **INSIDE(type)**: emit CONTINUE or END.

Table 1: All interfaces use the same examples and runtime capabilities.

Condition	Question isolated
Paired generic tags	Explicit bounded language-bus control
Fenced generic block	Familiar delimiter with Markdown collision risk
Label-only generic intent	Minimum generated control tokens; active task is payload
Transparent CPU trigger	Non-neural causal control baseline
Latent span controller	Parallel START/type/END control bus
Oracle span and type	Upper bound before R2/normalization/execution

Nesting and type switching are invalid. On END, generation pauses; the runtime extracts the naturally generated span, resolves and executes a capability, records provenance-bearing typed state, materializes a bounded result, and resumes OUTSIDE. BIO/BIES labels are retained as a matched alternative, not assumed superior.

3 Payload and Completion Ladders

The payload ladder starts with the marked ordinary-language span. If it is not directly executable, the runtime tries a deterministic parser, then a selected PRA/ICL capability skill, then short capability-specific normalization. A learned structured-argument head is justified only after those fail. This keeps span control distinct from normalization quality.

For END, compare neural prediction, deterministic parser completeness, their conservative AND, and a safe OR that fires only when either signal is high-confidence and policy permits. Arithmetic, date, and units may rely more on deterministic closure than RETRIEVE or HELP. Report premature and late termination separately.

4 Multi-Layer Control

Probe approximately $0.25L$, $0.40L$, $0.55L$, $0.70L$, $0.85L$, and L. Final-layer control is only the baseline. The registered aggregation ladder is: best development-selected layer, fixed mean of layer logits, majority or threshold vote, learned scalar layer weights, and a tiny MLP over layer logits. Hidden-state fusion is deferred unless logit aggregation fails. START and END may select different fixed mixtures, but every choice is frozen on development data before test evaluation.

5 Matched Benchmark and Baselines

Create one new causal-span freeze spanning calculator, date, units, graph, Datalog, retrieval, verification, ambiguous HELP, and NONE. Controls include quoted tags/fences, discussion of tools without a current need, ordinary prose, and long generations with multiple opportunities. Split by paraphrase family and namespace, not random surface form. Derive explicit and latent labels from the same examples.

5.1 Cross-paper public control registry

To avoid inventing a new task distribution, we freeze 280 redistribution-safe references from retained Papers 1.5, 2, and 2.5 selections: 40 each from GSM8K, BIG-bench Date Understanding, BIG-bench Unit Conversion, balanced ProofWriter, CLUTRR, CRAG, and TAT-QA. The artifact stores only source identifiers, benchmark IDs, content hashes, deterministic selection keys, coarse assistance type, and span-audit status. TAT-QA arithmetic/count rows record COMPUTE as a secondary action after primary RETRIEVE.

This registry is not yet a latent-control benchmark. Every row is marked `pending_audit`; a bounded FRAMES subset is also absent. Before any headline run, auditors must add earliest useful START, latest safe START, canonical END, payload span, assistance type, and first wrong or unsupported token where applicable. The manifest therefore records `headline_ready: false` and blocks result generation rather than synthesizing span labels from answers.

The required public matrix is four generic tools, paired tags, fenced blocks, label-only intent, CPU trigger, final-layer latent, multi-layer latent, tools-plus-watchdog, and oracle timing/type, all with identical R2 and backends.

The oracle-timed Paper 2 and 2.5 gateway audits establish only transport/schema parity; they do not substitute for voluntary or latent public runs. The registry now also hashes the Paper 1.5 CRAG condition matrix and Paper 2.5 public-suite readiness ledger. Those upstream artifacts validate oracle-timed generic transport but leave model timing, evidence execution, and final answers pending. They therefore add provenance, not a completed Paper 3 condition, and the manifest adds matched model-facing public results as an explicit blocker.

6 Training and Registry Invariance

The ladder is P0 frozen linear probes, P1 frozen multi-layer ensembles, P2 a small control-only LoRA if necessary, and P3 optional multitask LM/control training only after a positive gate. Train with calculator, date, and units in the registry; test R_1 unchanged after adding graph and Datalog, then later SMT, algebra, or retrieval backends. Score R_1 intent and R_2 capability selection separately. Adding an implementation must not silently alter test prompts or retrain the generic controller.

Result assimilation is a separate factorial axis: token versus latent control, crossed where feasible with base versus integration LoRA. A control gain must not be inferred from runtime-copy, and an integration gain must not be called a better interrupt.

7 Metrics and Gate

Report START/END precision, recall, and F1; exact span and token IoU; coarse type accuracy; NONE false-activation rate; premature/late END; payload parseability; R2 capability success; runtime and final exactness; controller parameters, FLOPs, and latency; explicit control tokens avoided; earliest reliable layer; cross-layer agreement and calibration; and registry-expansion deltas. Preserve raw causal traces and multi-seed uncertainty for headline comparisons.

Latent control advances only if it improves reliability or matched cost over generic explicit blocks and transparent CPU triggers without unsafe FAR or brittle END behavior. It is falsified as a needed mechanism if explicit generic blocks are equally reliable and cheap, aggregation adds no value, coupling is model-specific, or CPU triggers dominate. Early exit is exploratory only after strong multi-layer evidence and receives no headline claim in this version.

8 Scope, PRA, and Execution Order

There are no nested latent blocks, general planner, learned engine registry, joint base-model training, backtracking contribution, or silent promotion of uncertain language to fact. PRA is optional: intent may retrieve and materialize one to k capability skills, but the generic controller must work without it.

The execution order is dataset freeze; shared explicit/latent labels; final-layer probe; layer sweep; aggregation; BIO versus START/END; parser-assisted END; matched syntax and CPU comparisons; registry expansion; control LoRA only if needed; separate integration LoRA; and only then an early-exit study. Until retained artifacts complete these steps, this manuscript states hypotheses and protocol rather than empirical superiority.

The public registry can be reproduced from retained selections with:

```
python -m ccpu paper3 freeze-public \
  --compute-selection artifacts/paper2/public_benchmarks/data_v1/selection.jsonl \
  --crag-selection artifacts/paper1_5/public_benchmarks/crag_v1/data/selection.jsonl \
  --tatqa-selection artifacts/paper2_5/public_benchmarks/tatqa_v1/selection.jsonl \
  --crag-diagnostics artifacts/paper1_5/public_benchmarks/crag_v1/analysis/summary.json \
  --paper2-5-readiness artifacts/paper2_5/public_benchmarks/suite_v1/manifest.json \
  --per-benchmark 40 \
  --output-dir artifacts/paper3/public_control_v1
```

References

[1] T. Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS*, 2023.

[2] A. Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *ICLR*, 2024.

[3] L. Pan et al. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. *EMNLP*, 2023.